

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Казанский национальный исследовательский технический
университет им. А.Н. Туполева – КАИ»**

**Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности**

Лабораторная работа № 4

Дисциплина: «Аппаратные средства телекоммуникационных
систем»

Тема: «Вычисление значения с использованием функциональной
декомпозиции и критических секций»

Работу выполнила
студент гр. 4404
Миронихина К.И.

Казань, 2026

Используя функциональную декомпозицию перепишите задачу 5 для вычисления значения по формуле:

$$(\sum x + \sum y + \sum z) / 3N$$

где N – количество точек. Используйте глобальную сумму и критические секции.

Код:

```
#include <fstream>
#include <iostream>
#include <omp.h>
#include <vector>
#include <windows.h>
using namespace std;

int main() {
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    setlocale(LC_ALL, "Russian");

    int numThreads;
    cout << "Введите количество потоков: ";
    cin >> numThreads;

    if (numThreads <= 0) {
        cout << "Ошибка: число потоков должно быть положительным!" << endl;
        return 1;
    }

    // ===== ЧТЕНИЕ ФАЙЛА =====
    vector<double> Vx, Vy, Vz;
    double x, y, z;

    ifstream in("crd_big.txt");
    if (!in.is_open()) {
        cout << "Файл не открылся: crd_big.txt" << endl;
        return 1;
    }

    while (in >> x >> y >> z) {
        Vx.push_back(x);
        Vy.push_back(y);
        Vz.push_back(z);
    }

    in.close();

    int N = Vx.size();
    cout << "Прочитано точек: " << N << endl << endl;
```

```

double global_sum = 0;
omp_set_num_threads(numThreads);

double start = omp_get_wtime();

#pragma omp parallel sections
{
    #pragma omp section
    {
        double local = 0;
        for (int i = 0; i < N; i++) local += Vx[i];
    }
    #pragma omp critical
    global_sum += local;

    #pragma omp section
    {
        double local = 0;
        for (int i = 0; i < N; i++) local += Vy[i];
    }
    #pragma omp critical
    global_sum += local;

    #pragma omp section
    {
        double local = 0;
        for (int i = 0; i < N; i++) local += Vz[i];
    }
    #pragma omp critical
    global_sum += local;
}

double end = omp_get_wtime();

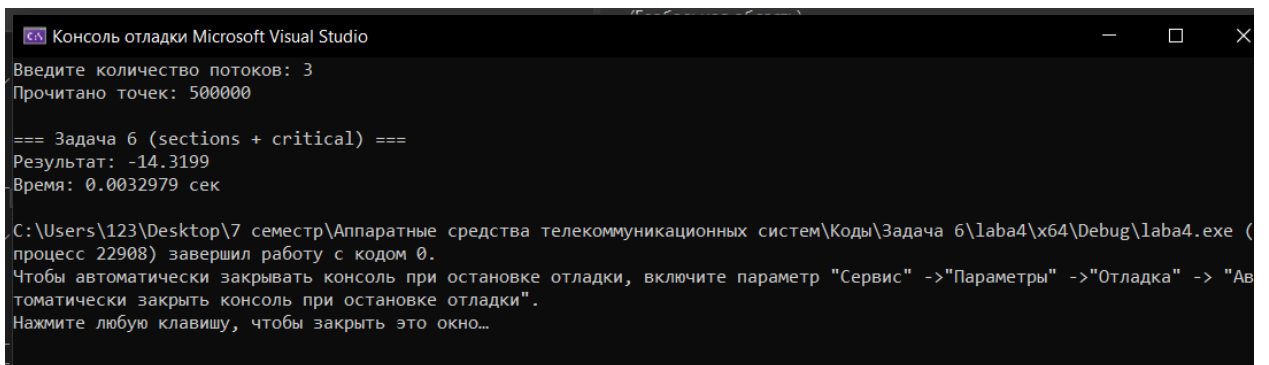
double result = global_sum / (3.0 * N);

cout << "=== Задача 6 (sections + critical) ===" << endl;
cout << "Результат: " << result << endl;
cout << "Время: " << end - start << " сек" << endl;

return 0;
}

```

Вывод:



```

Консоль отладки Microsoft Visual Studio
Введите количество потоков: 3
Прочитано точек: 500000

=== Задача 6 (sections + critical) ===
Результат: -14.3199
Время: 0.0032979 сек

C:\Users\123\Desktop\7 семестр\Аппаратные средства телекоммуникационных систем\Коды\Задача 6\laba4\x64\Debug\laba4.exe (
процесс 22908) завершил работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Ав
томатически закрывать консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно...

```

В этой версии используется директива `#pragma omp parallel sections` с тремя секциями — по одной на каждую координату. Результат собирается не в три отдельные переменные, а в одну общую `global_sum`. Первый поток берёт первую секцию и считает сумму всех `x` по всем точкам в локальной переменной `local`, затем один раз входит в критическую секцию `#pragma omp critical` и добавляет своё значение `local` к общей `global_sum`. Второй поток

делает то же самое для y , третий — для z . Критическая секция нужна потому, что все три потока пишут в одну переменную `global_sum` — без защиты возникла бы гонка, и часть слагаемых потерялась бы. Сначала потоки копят свои суммы локально, а потом одним куском добавляют их в общую переменную. В конце основной поток делит `global_sum` на $3N$ и получает одно число.